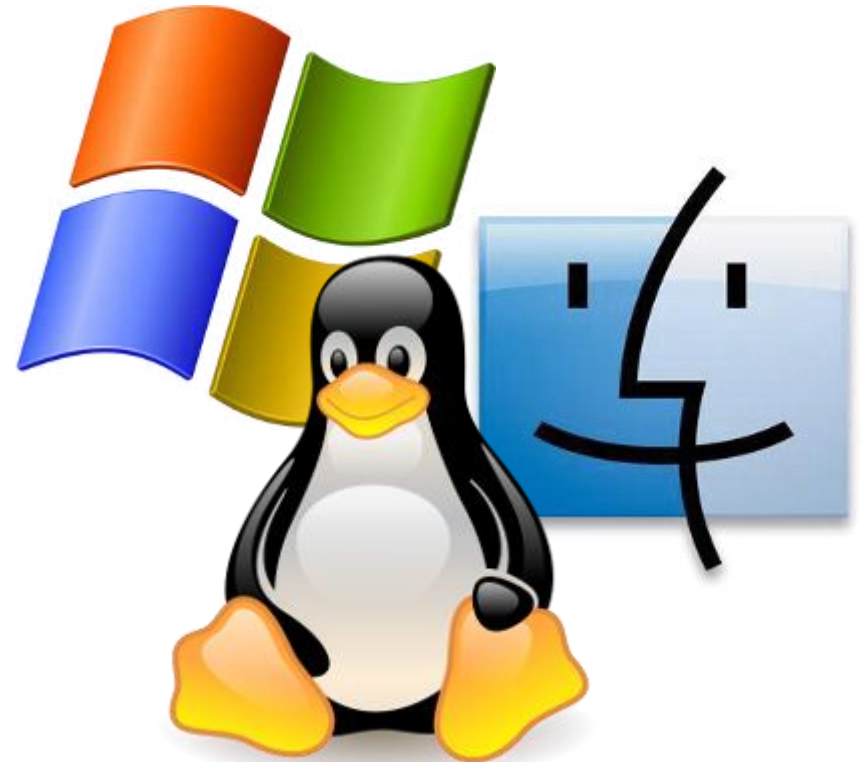


3 – Speicherverwaltung

Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

`tobias.werner.doz@leibniz-fh.de`

(in Teilen nach der Vorlesung
"Betriebssysteme" von Prof. Dominik Henrich
an der Universität Bayreuth)



[Logos von Windows 7, Linux und Mac OS X]

Einleitung

Heranführung

- Wie viel Speicher gibt es in Ihrem Computer?
- Was ist ein CPU-Register?
- Was macht eine virtuelle Maschine?
- Was ist eine Auslagerungsdatei bzw. eine Swap-Partition?

Kompetenzen

- einen Einblick in die Speicherarchitektur üblicher Computersysteme erhalten
- die Speicherverwaltung moderner Betriebssysteme verstehen
- zeitliche und räumliche Zuteilungsverfahren begrenzter Ressourcen erlernen

Vortragsübersicht

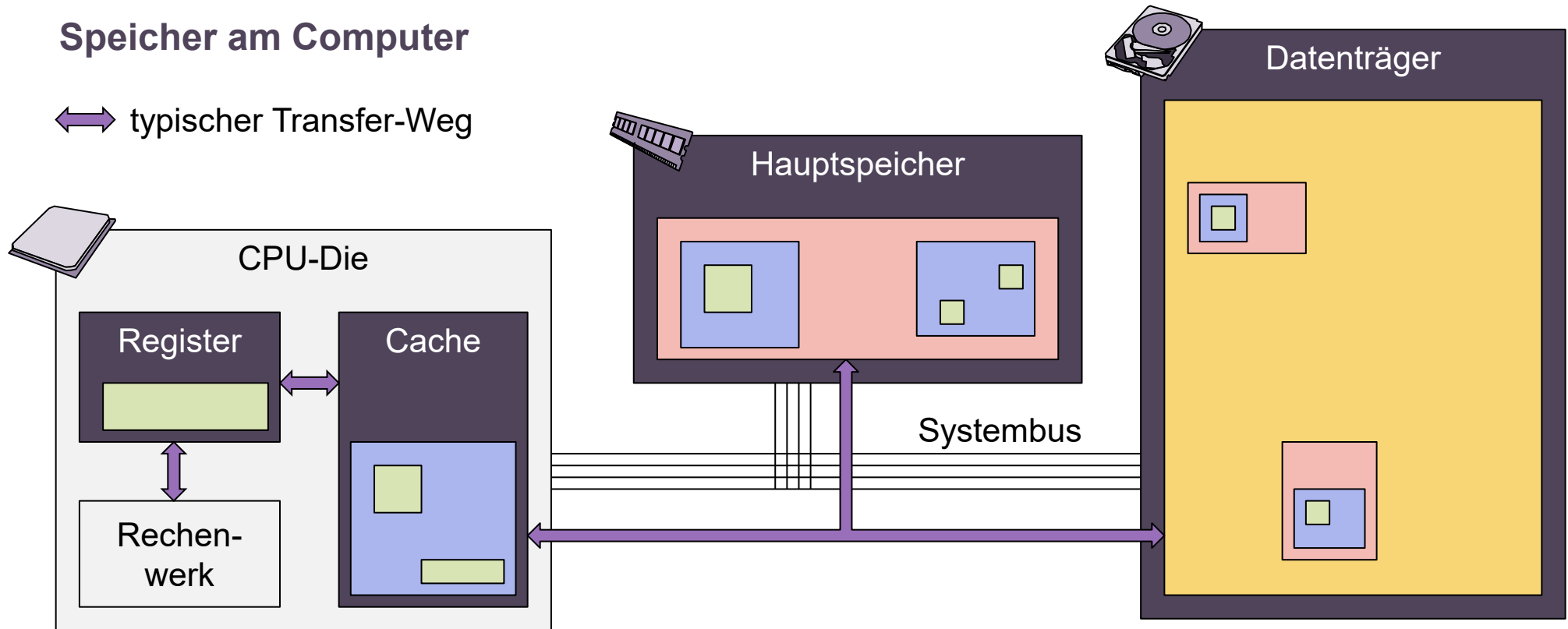
3 – Speicherverwaltung

- Speicherarchitektur
- Die vier Probleme der Speicherverwaltung
- Virtueller Speicher
- Behandlung von Seitenfehlern

Speicherarchitektur (1)

Speicher am Computer

↔ typischer Transfer-Weg



- Computerspeicher ist hierarchisch strukturiert
 - Speicher "näher" am Rechenwerk ist schneller, aber teurer und deshalb kleiner
 - "weiter" entfernt liegender Speicher ist langsamer, aber günstiger und daher größer
- Datenstücke müssen bei Bedarf transferiert und/oder synchronisiert werden

Speicherarchitektur (2)

Speichereinheiten auf dem CPU-Die

Stand für alle Angaben: ca. 2020



CPU-Register:

- sehr kleine, sehr schnelle Speichereinheiten mit direkter Anbindung an das Rechenwerk
- werden als direkte Eingabe und Ausgabe für CPU-Berechnungen genutzt
- Verwaltung und Synchronisation typischerweise durch OS und laufende Programme
- Speicherplatz: ca. 100 bis ca. 1000 Byte pro Kern
(z. B. x64-Architektur: ca. 50 bis 100 sinnvoll nutzbare Register mit je 8 Byte)



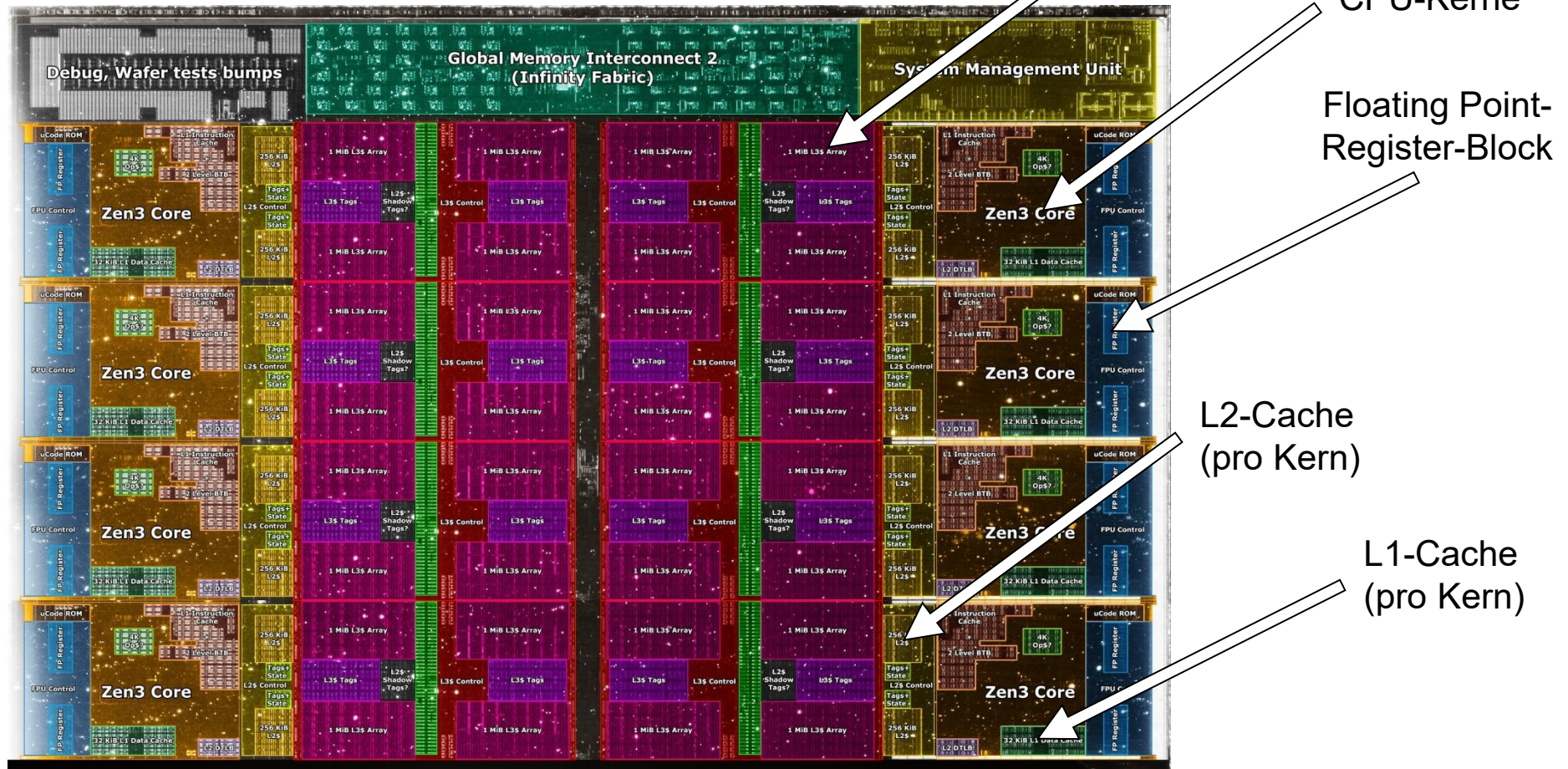
CPU-Caches:

- schnelle, eher kleine Zwischenspeicher für Daten aus dem Hauptspeicher
- in mehreren Ebenen verfügbar (z. B. L1- bis L3-Caches)
- Verwaltung und Synchronisation in der Regel automatisch über CPU
- Zeit zum Laden in Register: ca. 1 bis ca. 5 ns
- Speicherplatz: ca. 1 kB bis ca. 100 MB pro Kern oder geteilt
(z. B. AMD Ryzen 7600: 384 kB L1 pro Kern, 6 MB L2 pro Kern, 32 MB L3 geteilt)

Speicherarchitektur (3)

Beispiel CPU-Aufbau

AMD Ryzen 5000 (Zen3-Architektur), 2020

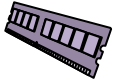


[<https://wccftech.com>]

Speicherarchitektur (4)

Speichereinheiten mit Bus-Anbindung

Stand für alle Angaben: ca. 2020



Hauptspeicher:

- großer, langsamer Speicher für Daten, auf denen eventuell bald gerechnet werden muss
- flüchtig
- Zuteilung durch OS, direkter Zugriff durch Programmanweisungen
- Zeit zum Laden in Register: ca. 5 bis ca. 20 ns
- Speicherplatz: ca. 4 GB bis ca. 256 GB

(z. B. Samsung Galaxy A40: 4 GB, Kingston Desktop-RAM: bis 64 GB pro Modul)



Datenträger:

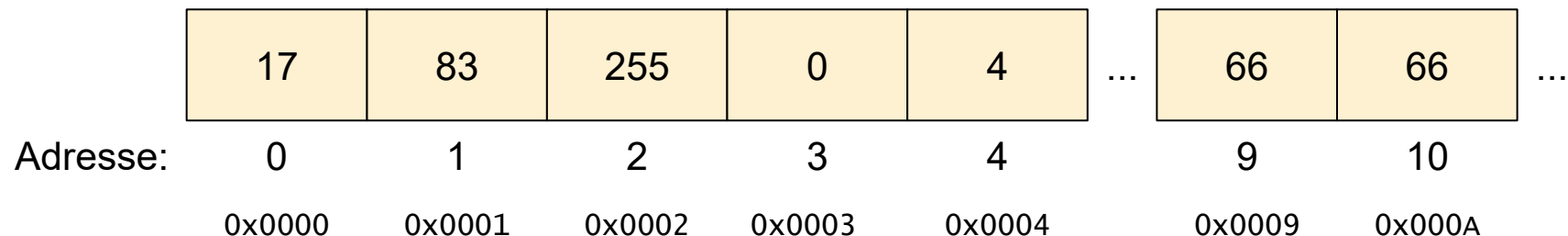
- sehr großer, sehr langsamer Speicher für Daten, die nicht direkt gebraucht werden
- nicht-flüchtig
- Zuteilung durch OS, indirekter Zugriff über Systemaufrufe
- Zeit zum Laden in Register: ca. 5 bis ca. 10 ms
- Speicherplatz pro Datenträger: ca. 100 GB bis 20 TB

(z. B. Western Digital SN850X-SSD bis 4 TB, Toshiba MG10-Festplatte bis 20 TB)

Speicherarchitektur (5)

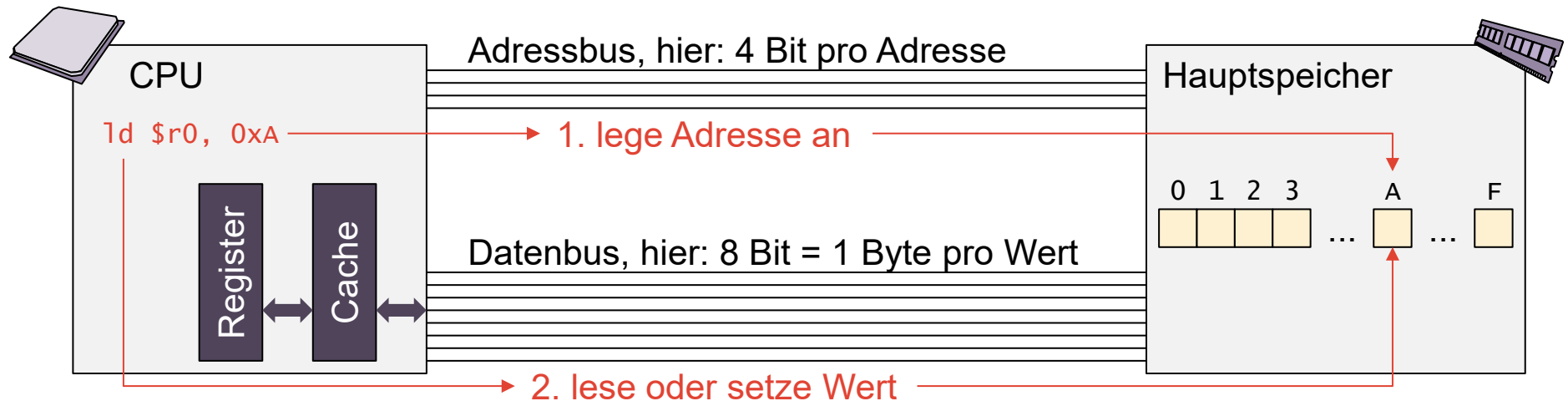
Hauptspeichermodell für Betriebssysteme und Programme

- Hauptspeicher besteht aus einer Folge von Speicherzellen
- Speicherzellen sind von 0 an lückenlos aufsteigend nummeriert
 - *Adresse* ist die eindeutige Nummer jeder Speicherzelle
 - Darstellung als Hexadezimalzahl (z. B. mit 0x oder h)
(z. B. 0xC1AABDCB oder C1AABDCBh statt 3249192395)
- jede Speicherzelle ist exakt ein Byte groß
 - jede Speicherzelle kann *Wert* von 0 bis 255 speichern
 - die Bits in einem Byte haben selbst **keine** Adresse
- Wert und Adresse sind im Allgemeinen nicht gleich
 - vgl. Konto: Kontonummer ≈ Adresse, Kontostand ≈ Wert



Speicherarchitektur (6)

Speicherzugriff nach dem Speichermodell

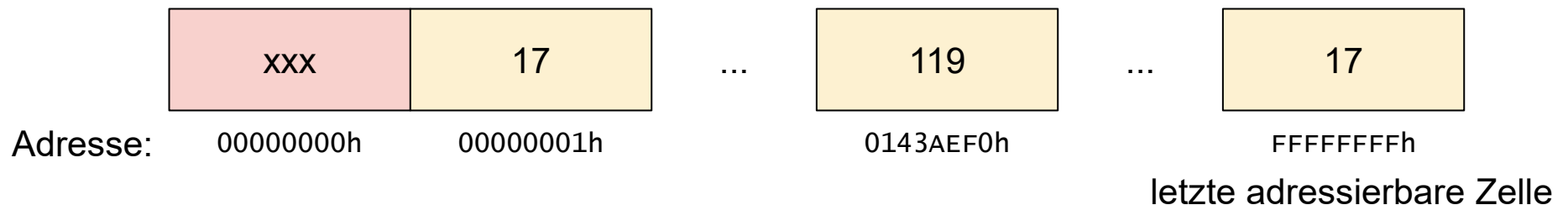


- allgemein: ein *Bus* (Wortherkunft unklar) ist ein digitaler Kommunikationskanal
- CPU nutzt Maschinenbefehle, um Werte zwischen Speicherzellen und Registern hin- und herzukopieren
 - naive, veraltete Realisierung über einen *Adressbus* und einen *Datenbus*:
CPU wählt über Adressbus die Adresse der gewünschten Speicherzelle, dann lässt sich über den Datenbus die Speicherzelle lesen oder verändern
 - Zahl der Leitungen im Adressbus gibt Rahmenbedingungen für Adressen vor (d. h. feste Bit-Breite n für n -Bit-OS und maximal mögliche Adresse)

Speicherarchitektur (7)

Rahmenbedingungen des Speichermodells

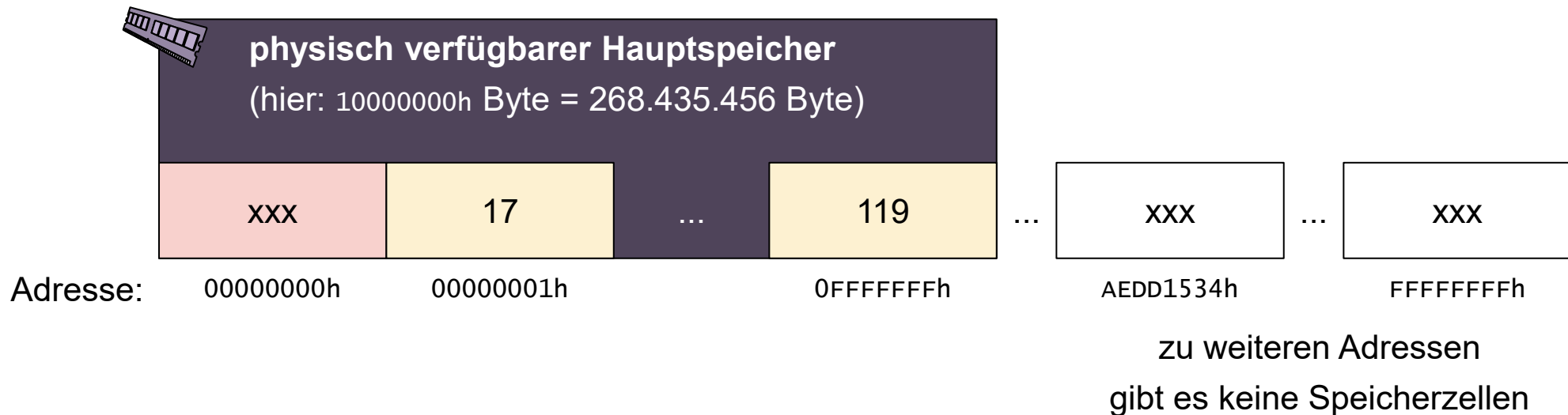
- *adressierbarer Hauptspeicher*:
 - in einem Maschinenbefehl bzw. per Bus maximal ansprechbare Speicherzellen (für die VL-Abbildungen: 2^{32} Speicherzellen, x64-Architektur: 2^{64} Speicherzellen, in Übungsaufgaben tendenziell $2^6 \dots 2^{12}$ Speicherzellen)
 - korrespondiert mit der Bit-Breite von Adressen:
Adressen mit b Bits heißt 2^b Byte adressierbarer Hauptspeicher
- in der Speicherzelle zu Adresse 0 legt Software üblicherweise kein Datum ab
 - Verwendung als Marker (`nullptr`, `NULL`, `null`) für Verweise ohne Ziel
- vom OS nicht kontrollierbare Umstände (z. B. Caches, Verteilung auf RAM-Module) werden im Speichermodell nicht berücksichtigt



Speicherarchitektur (8)

Abbildung auf die Realität

- *physisch verfügbarer Hauptspeicher* (kurz *physischer Hauptspeicher*):
 - begrenzte Zahl an tatsächlich im verbauten Hauptspeicher verfügbare Speicherzellen
 - typische 64-Bit-Systeme haben 4 bis 32 GB physisch verfügbaren Hauptspeicher, theoretisch adressierbar wären 9 EB (ca. 9 Millionen GB)
 - 32-Bit-Systeme haben höchstens 2 bis 4 GB physisch verfügbaren Hauptspeicher, mehr Speicher ließe sich wegen begrenzten Adressen nicht ansprechen
 - die Adressbreite bleibt unabhängig der physisch verbauten Speichermenge gleich



Speicherarchitektur (9)

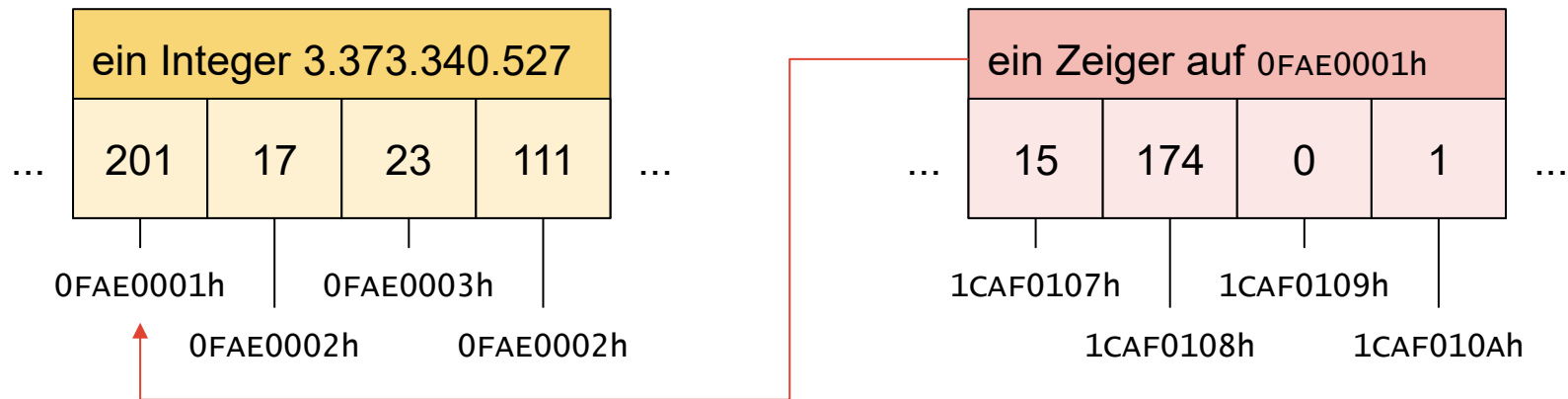
Variablen und Verweise im Speichermodell

Idee *Wertvariable*:

fasse mehrere, aufeinanderfolgende Speicherzellen zu einem semantischen Wert zusammen
(z. B. belegt eine `int`-Variable vier aufeinanderfolgende Speicherzellen)
→ aktuelle Hardware kann in einem Maschinencodibefehl statt einzelne Speicherzellen solche Werte über den Bus übertragen und in den Registern verrechnen

Idee *Zeigervariable* (manchmal *Verweis*, *Referenz*):

eine spezielle Wertvariable, die sich die Adresse einer anderen Variablen merkt
→ bei Variablen mit mehreren Speicherzellen die erste, zugehörige Adresse



Vortragsübersicht

3 – Speicherverwaltung

- Speicherarchitektur
- Die vier Probleme der Speicherverwaltung
- Virtueller Speicher
- Behandlung von Seitenfehlern

Die vier Probleme der Speicherverwaltung (1)

Speicherzuteilung an Prozesse

gegeben:

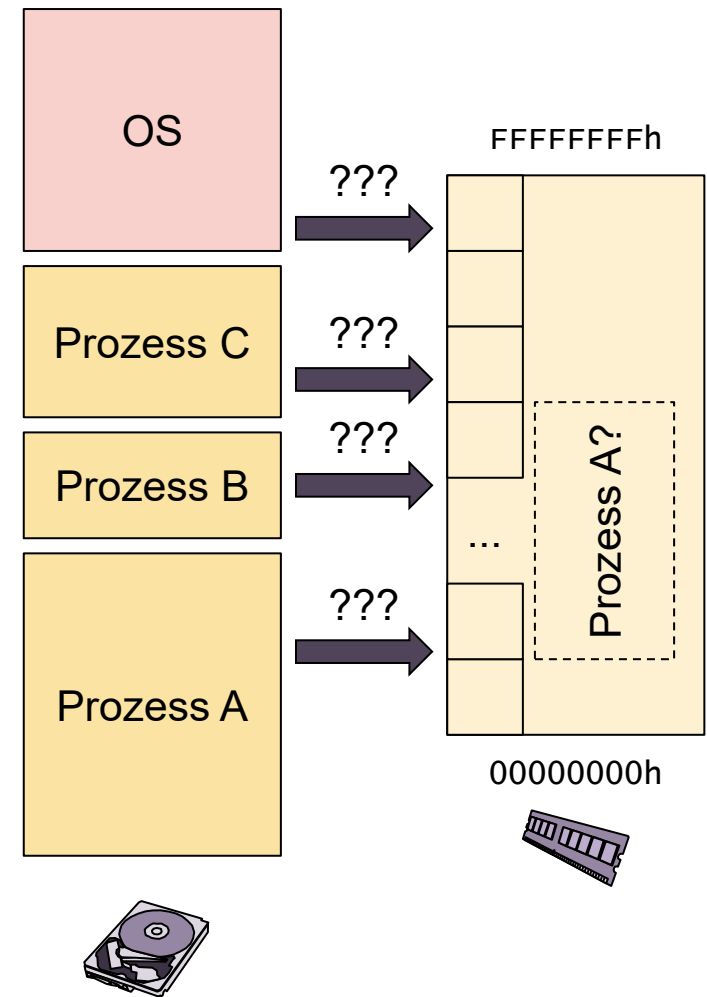
- adressierbarer Hauptspeicher
- gleichzeitig abzuarbeitende Prozesse (i. d. R. als Software auf Datenträger)

gesucht:

- dynamische Zuteilung (d. h. Allokation) vom adressierbaren Hauptspeicher an die laufenden Prozesse

Rahmenbedingungen:

- Prozesse brauchen ihren Speicherplatz am Stück
- Prozesse brauchen evtl. mehr Speicherplatz als im Hauptspeicher physisch verfügbar
- Lösung der *vier Probleme der Speicherverwaltung* (siehe nächste Folien für Details)

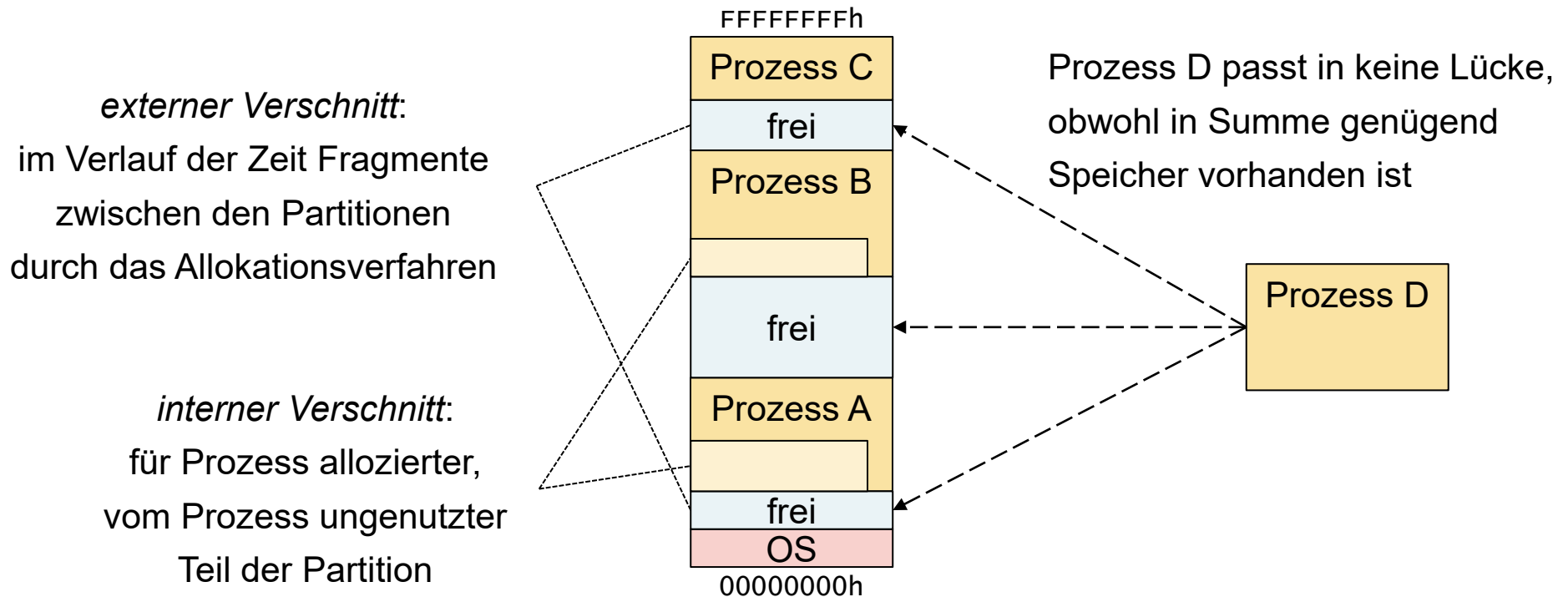


Die vier Probleme der Speicherverwaltung (2)

1. Problem: *Speicherfragmentierung*

Annahme: weise jedem Prozess eine *Partition* (d. h. Speicher am Stück) zu

Problem: bei ungünstiger Zuteilung (z. B. über längere OS-Laufzeit) zerfällt der Hauptspeicher in *Fragmente* (kleine Stücke), die in Summe zwar genug Speicher für weitere Prozesse haben, aber nicht am Stück

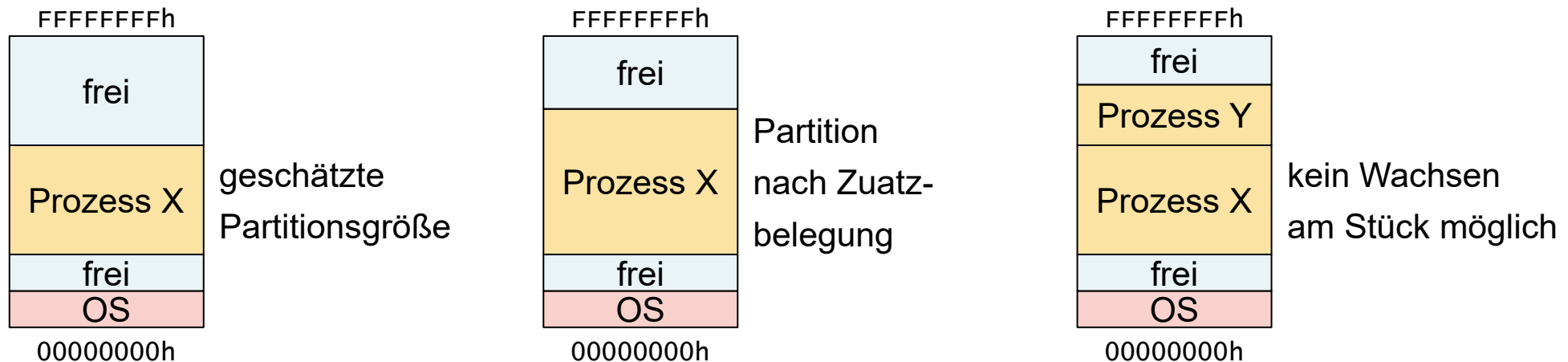


Die vier Probleme der Speicherverwaltung (3)

2. Problem: *Speicherzusatzbelegung*

Annahme: jeder Prozess erhält beim Start eine Partition fester Größe

Problem: Prozesse wissen im Voraus meist nicht, wie viel Speicher sie benötigen
(z. B. wegen dynamisch wachsenden Datenstrukturen je nach Eingabe)
→ Partitionen müssen nachträglich wachsen können



naive Lösungsansätze:

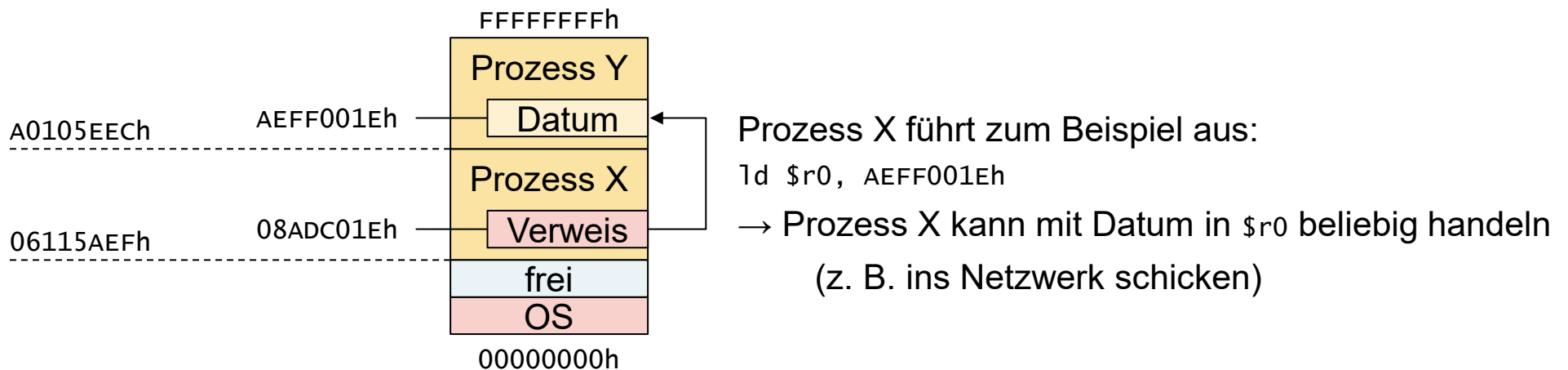
- für jeden Prozess wird vorsorglich viel mehr Speicher belegt
- Prozesspartitionen können in freien, benachbarten Speicher wachsen
- Prozesse auf Datenträger auslagern und an anderer Stelle einlagern

Die vier Probleme der Speicherverwaltung (4)

3. Problem: *Speicherschutz*

Annahme: erlaube Prozessen direkten Lese- und Schreibzugriff auf alle Speicherzellen

Problem: Prozesse können durch Fehler oder absichtlich fremde Daten lesen oder ändern



naive Lösungsansätze:

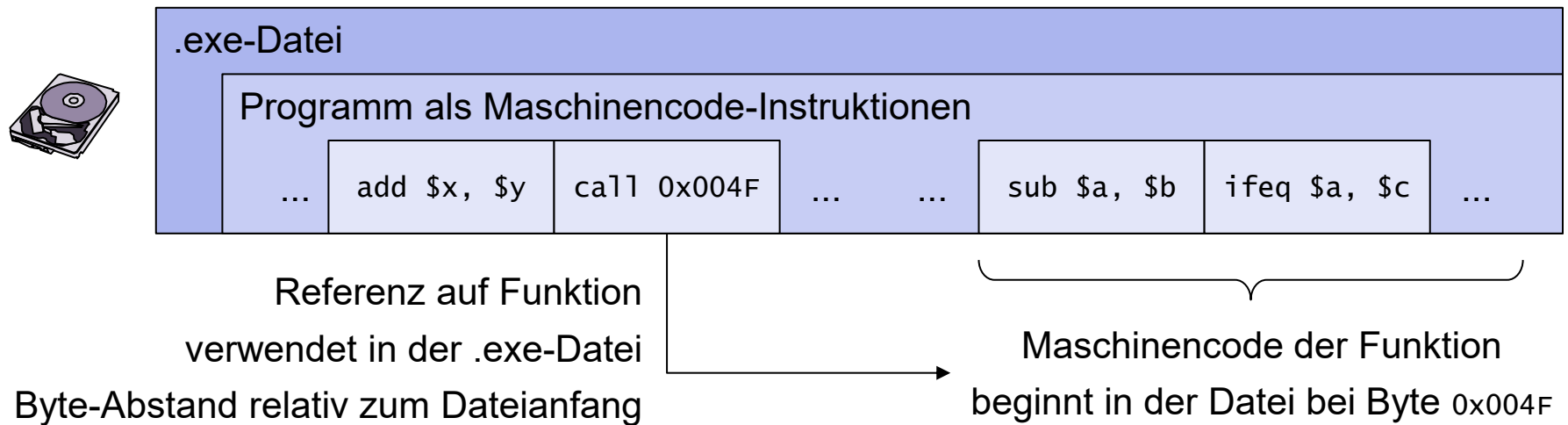
- Speicherzugriff nur über verhältnismäßig langsame Systemaufrufe
- vom OS gesetzte CPU-Schutzcodes ordnen Prozessen Speicherbereiche zu
- Zugriffsbeschränkung mit geprüften Basis-/Grenz-Registern

Die vier Probleme der Speicherverwaltung (5)

4. Problem: *Relokation*

Annahme: Prozess modelliert Verweise (z. B. auf Variablen oder Funktionen) mittels Adressen (vgl. frühere Folien)

Problem: Verweise gehen beim Verlagern der Prozesspartition im Speicher genau wie beim Starten oder Laden von einem Datenträger kaputt



naive Lösungsansätze:

- bei Prozessstart und Verlagerung alle Verweise entsprechend modifizieren
- Adressmodifikation mit geprüften Basis-/Grenz-Registern

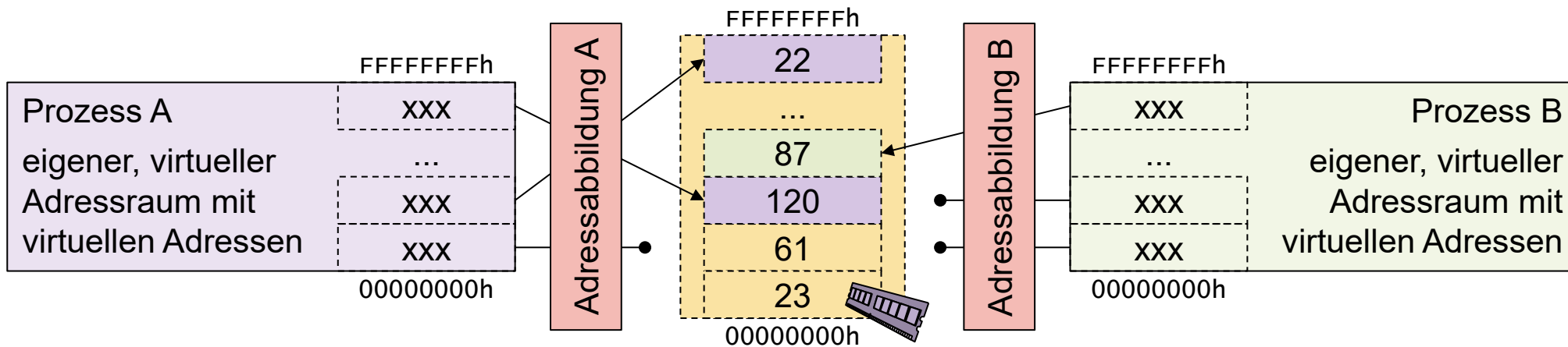
3 – Speicherverwaltung

- Speicherarchitektur
- Die vier Probleme der Speicherverwaltung
- Virtueller Speicher
- Behandlung von Seitenfehlern

Virtueller Speicher (1)

Virtuelle Speicherverwaltung (kurz virtueller Speicher)

- jeder Prozess erhält ein eigenes Speichermodell mit *virtuellen Adressen* (d. h. pro Prozess neu durchzählende Adressen ohne echte Speicherzellen)
- eine *Adressabbildung* (d. h. Algorithmus mit Datenstruktur) übersetzt zwischen virtuellen Adressen und *physischen Adressen* (d. h. zu echten Speicherzellen)



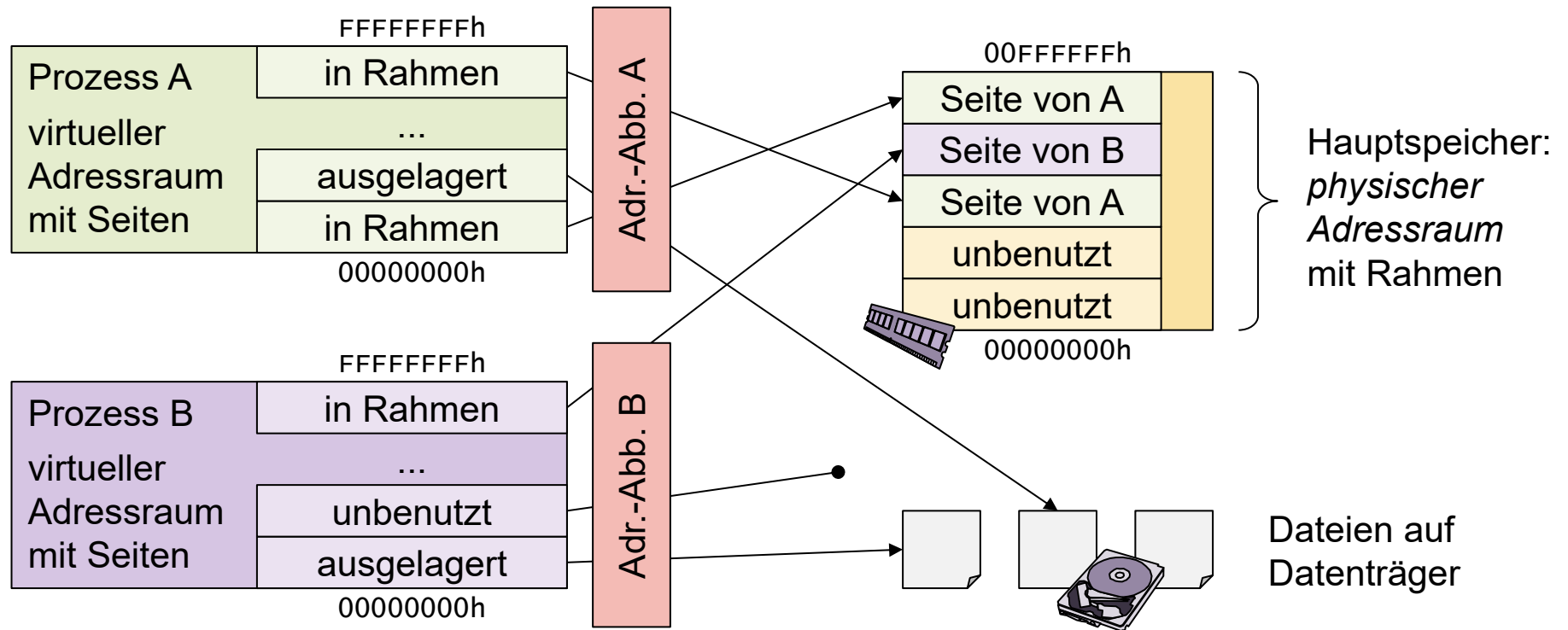
Folgerungen:

- es gibt virtuelle bzw. physische Adressen ohne physische bzw. virtuelle Adresse
- zur selben virtuellen Adresse können zwei Prozesse verschiedene Daten speichern

Virtueller Speicher (2)

Optimierung der Adressabbildung

- Problem:** individuelle Abbildung einzelner Adressen ist zu teuer (Laufzeit, Speicher)
- Lösung:**
- zerlege virtuellen Adressraum vollständig in (*Speicher-*)*Seiten* fester Größe
 - zerlege Hauptspeicher vollständig in (*Speicher-*)*Rahmen* gleicher Größe
 - bilde Seiten auf Rahmen oder Sonderfälle (z. B. ungenutzt, Auslagerung) ab

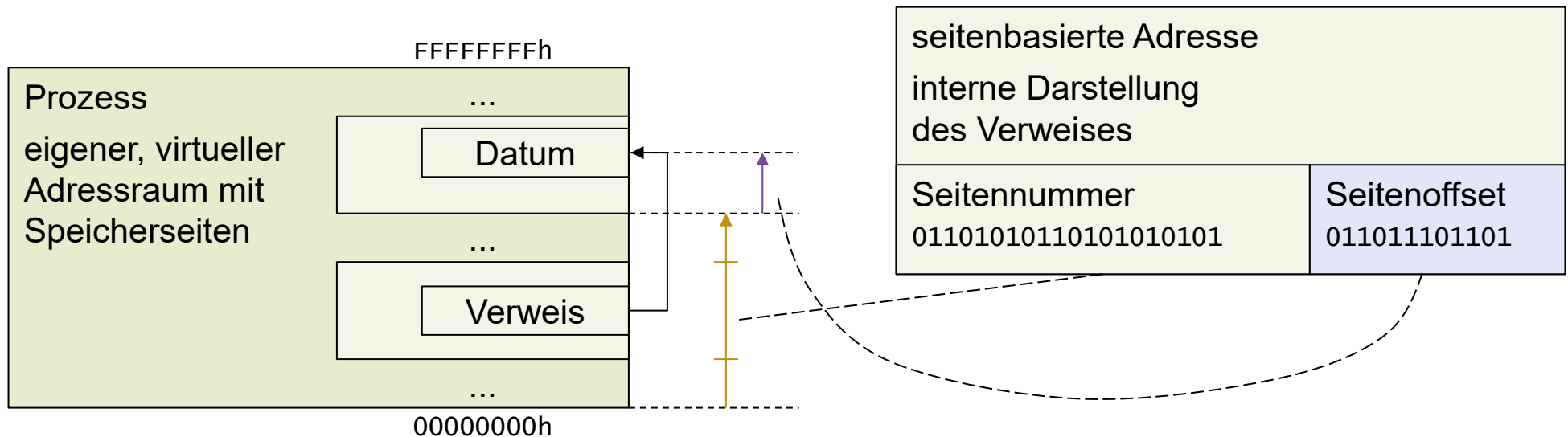


Virtueller Speicher (3)

Umsetzung mit seitenbasierten Adressen

seitenbasierte Adresse:

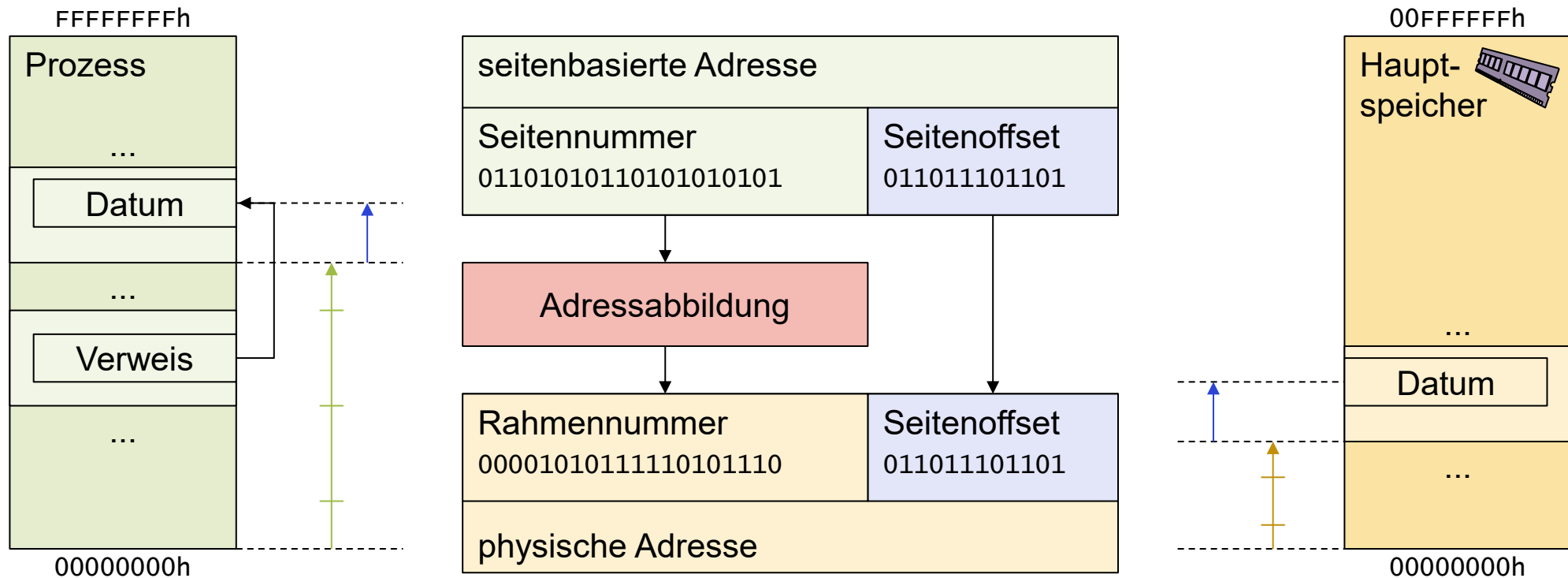
- zerlege jede virtuelle Adresse (z. B. 4 Byte, z. B. aus einem internen Verweis)
in eine *Seiten-Nummer* (z. B. 20 Bit) und in einen *Seiten-Offset* (z. B. 12 Bit)
- Seiten-Nummer ist Abstand der Seite (in Seiten) vom Anfang des Adressraums
- Seiten-Offset ist byte-weiser Abstand zwischen Seiten-Anfang und Verweisziel



Virtueller Speicher (4)

Adressabbildung

Vorgehen: ersetze Seiten-Nummer bei jedem Zugriff durch aktuell zugeordnete Nummer des Rahmens im Hauptspeicher und behalte Seiten-Offset unverändert bei



Folgerungen:

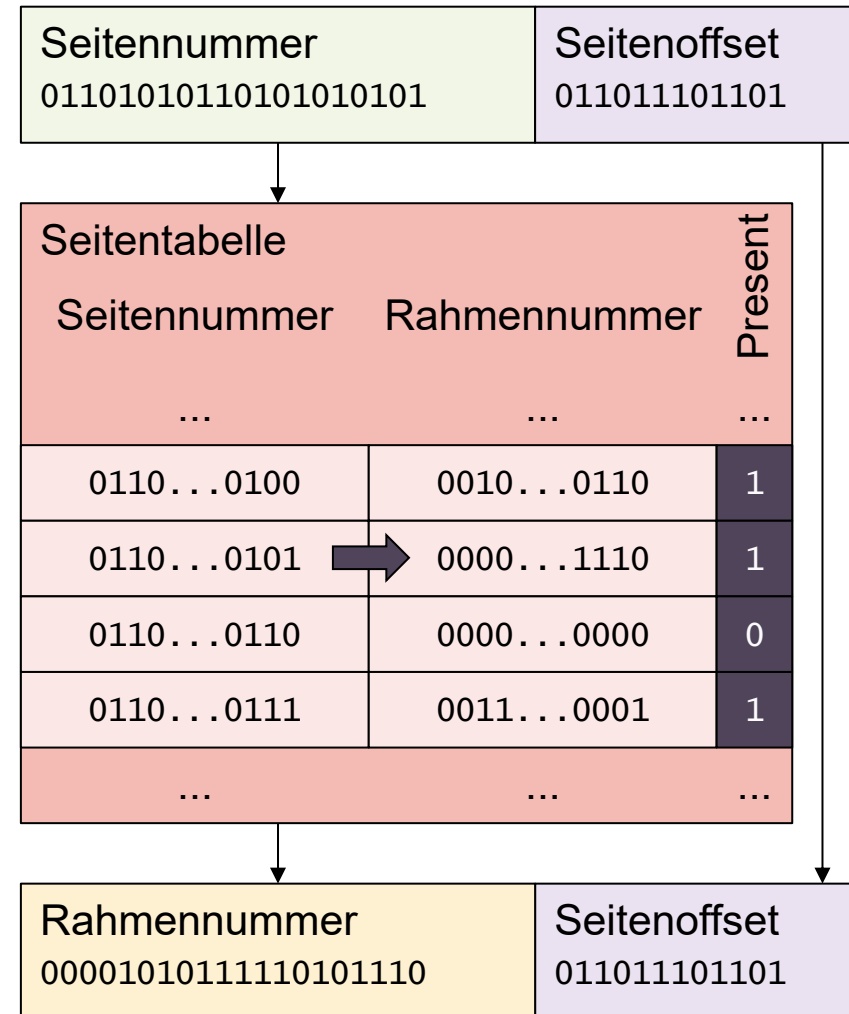
- Seiten- und Rahmen-Nummern müssen die gleiche Bit-Breite besitzen
- es gibt i. d. R. Rahmen-Nummern, zu denen kein Rahmen existiert

Virtueller Speicher (5)

Implementierung der Adressabbildung

einfache Seitentabelle pro Prozess:

- speichere für jede virtuelle Seitennummer die aktuell zugeordnete Rahmennummer in nach Seitennummer sortierter Tabelle
→ schlage mit virtueller Seitennummer nach und ersetze die Seitennummer in Adresse durch die Rahmennummer
- zusätzliches *Present*-Bit gibt an, ob die Seite aktuell auf Datenträger ausgelagert ist
→ Zugriff auf gerade ausgelagerte Seite (Present-Bit 0) verursacht *Seitenfehler*
→ OS lädt die Seite relativ zeitaufwendig vom Datenträger nach



Virtueller Speicher (6)

Erweiterte Adressabbildung

zusätzlich in Seitentabelle geführte Daten:

- Prozess-ID → Information darüber, wem diese Seitentabelle gehört
- Present-Bit → Seite ist gerade im Hauptspeicher statt auf Datenträger ausgelagert
- Reference-Bit → gesetzt, falls auf Daten der Seite kürzlich zugegriffen wurde
- Modify-Bit → gesetzt, falls auf Seite seit Einlagerung in Rahmen geschrieben wurde
- Schutz-Modus → Marker für schreibbeschränkte Seite (z. B. nur durch Systemaufruf)
- ...

Bewertung:

- einfache Seitentabellen sind kostengünstig anzuwenden (Zeigerarithmetik!)
- einfache Seitentabellen brauchen Platz für alle (d. h. selbst unbelegte) Seiten
→ hoher Speicherbedarf für jeden einzelnen Prozess

Lösung:

- erweiterte Seitentabellen, z. B. hierarchische oder invertierte Seitentabellen
(→ übersteigt VL-Umfang)

Virtueller Speicher (7)

Vollständiges Beispiel

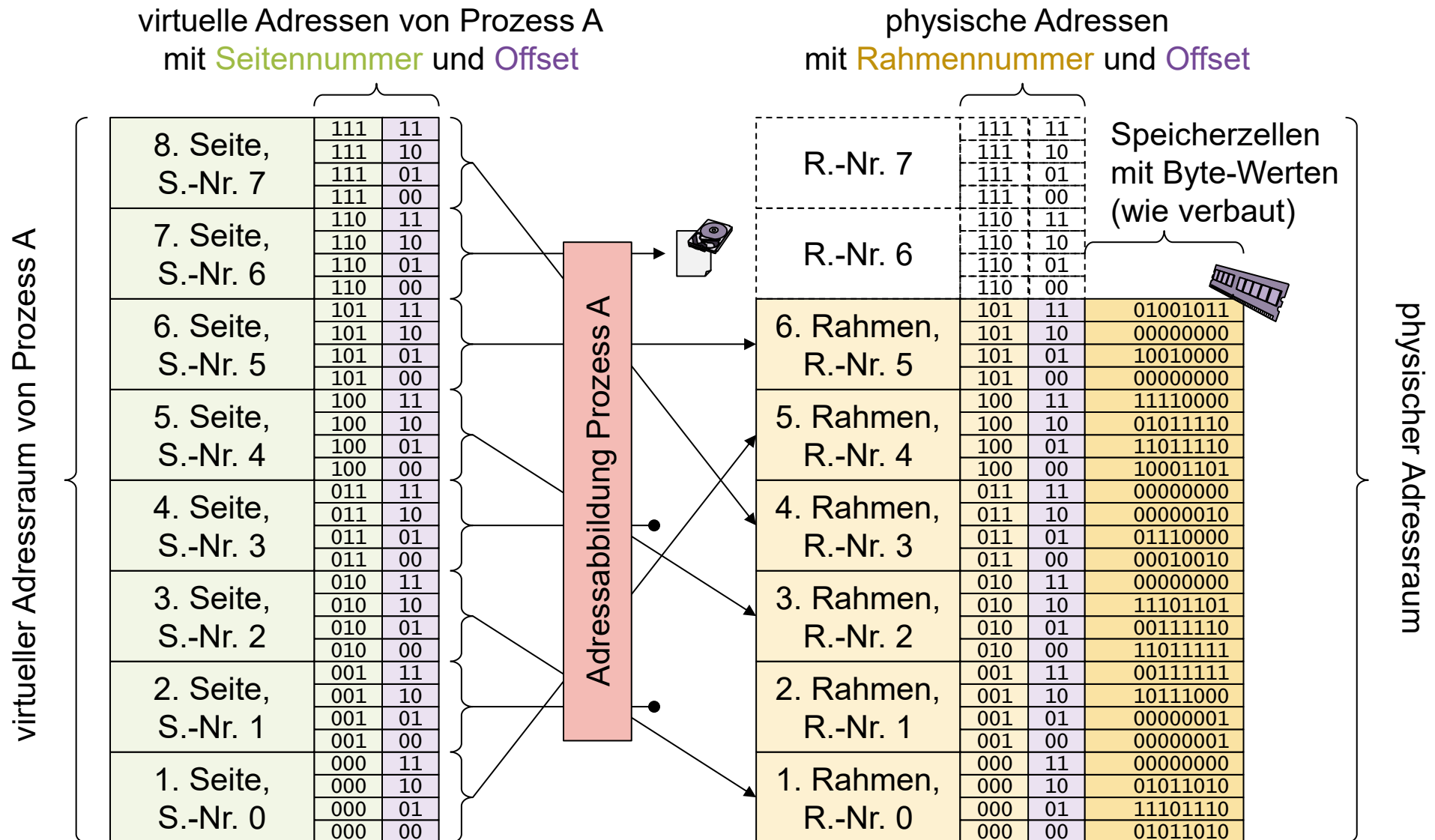
gegeben:

- Minirechner mit 5-Bit-Betriebssystem und 24 Byte Hauptspeicher
- virtuelle Adressierung mit 3 Bit großen Seitennummern

exemplarische Folgerungen:

- virtuelle Adressen sowie physische Adressen haben **immer exakt** 5 Bit
- es gibt **exakt** 2^5 virtuelle Adressen
 - der virtuelle Adressraum **jedes** Prozesses ist **exakt** 2^5 Byte = 32 Byte groß,
da man jeder virtuellen Adresse eine Speicherzelle (d. h. ein Byte) zuordnen kann
- es gibt **exakt** 2^5 physische Adressen
 - der (physische) Hauptspeicher ist **höchstens** 2^5 Byte = 32 Byte groß,
da es pro physischer Adresse eine Speicherzelle (d. h. ein Byte) geben kann,
es kann aber weniger Speicher als möglich verbaut sein
- für das Offset auf Seite bzw. Rahmen bleiben 5 Bit – 3 Bit = 2 Bit
- jede Seite beinhaltet 2^2 virtuelle Adressen und daher 2^2 Byte = 4 Byte
- jeder Rahmen beinhaltet 2^2 physische Adressen und daher 2^2 Byte = 4 Byte

Virtueller Speicher (8)



Virtueller Speicher (9)

erweiterte Seitentabelle zu Prozess A für Beispielbild:

Seitennummer	Rahmennummer	Present	Used	Read-Only
000	100	1	1	1
001	000	0	0	0
010	000	1	1	0
011	000	0	0	0
100	010	1	1	0
101	101	1	1	0
110	000	0	1	0
111	011	1	1	0

- Used-Bit: gesetzt, wenn auf der Seite ein Datum gespeichert wurde
- Read-Only-Bit: falls gesetzt, führen Schreibzugriffe auf Seite zu Prozessabbruch

exemplarische Folgerungen:

- Lesebefehl auf virtuelle Adresse **11110** liest von physischer Adresse **01110**
- Schreibbefehl auf virtuelle Adresse **00000** schreibt auf physische Adresse **10000**
→ Zugriffsfehler, von Seite darf nur gelesen werden (vgl. null-Adresse)
- Schreibbefehl auf virtuelle Adresse **11011** schreibt auf physische Adresse **00011**
→ Seitenfehler, Seite liegt gerade auf Datenträger, Seite nachladen

3 – Speicherverwaltung

- Speicherarchitektur
- Die vier Probleme der Speicherverwaltung
- Virtueller Speicher
- Behandlung von Seitenfehlern

Behandlung von Seitenfehlern (1)

Grundsätzlicher Allokationsprozess

- jeder Prozess X erhält beim Start vom Betriebssystem ein paar Seiten vorbereitet:
das Betriebssystem schreibt grundlegende Daten in Speicherzellen bzw. Rahmen und verknüpft diese auf geeignete Art und Weise mit der Seitentabelle von X (z. B. für Maschinencode des Prozesses, für den Stack des Hauptthreads)
- der Prozess X führt seinen Maschinencode nach Scheduling durch Betriebssystem aus, damit gibt es zwei wesentliche Anwendungsfälle:
 - der Prozess X fordert per Systemaufruf (z. B. `malloc`) Speicher an, das Betriebssystem weist X dann neue Rahmen und Seiten zu, passt die Seitentabelle an und nennt X jene virtuelle Adressen, die zu der neuen Speicherallokation gehören
 - der Prozess X greift per Maschinencode (z. B. `ld $r0, 0xAEF30FF7`) auf eine virtuelle Adresse zu, wobei die CPU mit der Seitentabelle den Zugriff auf die zugeordnete physische Speicherzelle vornimmt oder bei einem Seitenfehler das Betriebssystem benachrichtigt

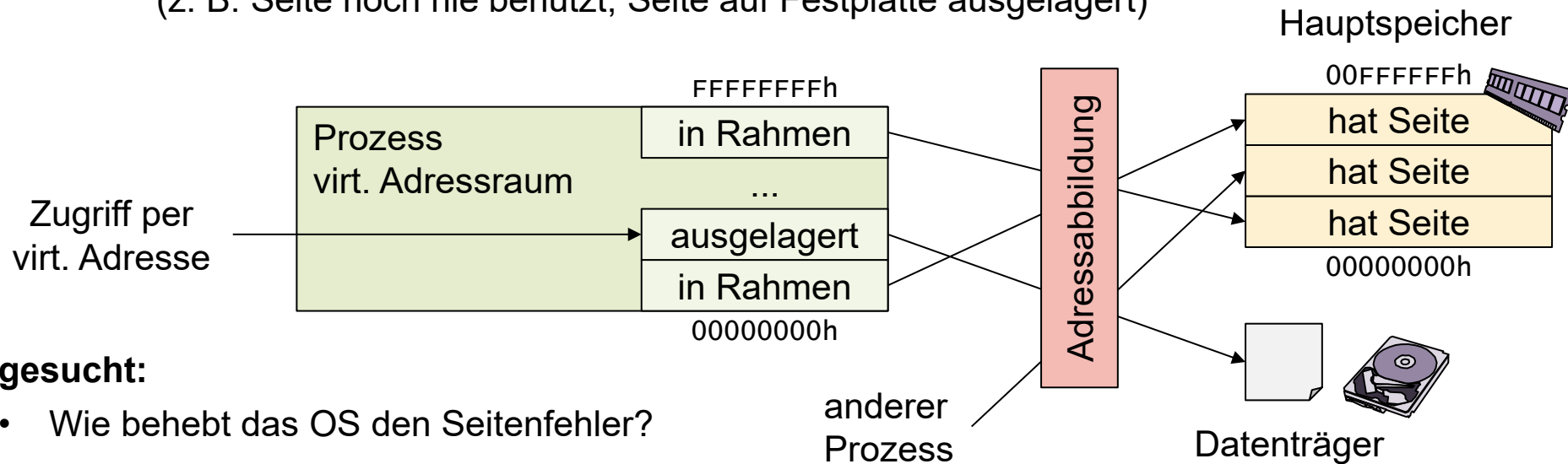
Behandlung von Seitenfehlern (2)

Problemstellung

gegeben:

- virtuelle Adressierung wie zuvor
- ein *Seitenfehler* passiert:

Maschinencode greift auf eine Seite zu, der kein Rahmen zugewiesen ist
(z. B. Seite noch nie benutzt, Seite auf Festplatte ausgelagert)



gesucht:

- Wie behebt das OS den Seitenfehler?
- Welchen Rahmen weise ich der Seite zu?
- Wenn alle Rahmen mit Seiten belegt sind, welche der Seiten lagere ich aus?
→ lagere ich häufig benutzte Seiten aus, muss ich diese häufig wieder nachladen

Behandlung von Seitenfehlern (3)

OS- und hardwareseitige Behandlung von Seitenfehlern

dazu sind 10 Schritte nötig:

1. CPU bemerkt bei Zugriff den Seitenfehler, sichert Prozess-Instruktionszähler und löst Kernel-Trap aus → von Hardware angestoßener Systemaufruf
2. OS sichert aktuelle Registerinhalte des laufenden Prozesses
3. OS bestimmt aufgerufene, virtuelle Seite für Nachschlagen in Seitentabelle (z. B. über spezielles Register oder Analyse der letzten Maschinenbefehle)
4. OS bestimmt und prüft Zugriffsrechte auf virtuelle Seite und sucht freien Seitenrahmen im Hauptspeicher
5. OS suspendiert einen anderen Prozess X, falls dessen Seite verdrängt wird, und sichere Seiteninhalt auf Datenträger
6. OS liest angeforderte Seite vom Datenträger in den Seitenrahmen
7. OS aktualisiert Seitentabelle mit Zuordnung zu Rahmen
8. OS setzt Prozess-Instruktionszähler auf Befehl, der Seitenfehler verursacht hat
9. OS versetzt gegebenenfalls suspendierten, anderen Prozess X in Bereit-Zustand
10. OS restauriert Prozess, setzt Ausführung zurück, wechselt Prozess auf bereit

Prozesswechsel zu anderem,
bereiten Prozess möglich

Behandlung von Seitenfehlern (4)

Verdrängungsstrategie

Aufgabenstellung:

Wenn alle Rahmen belegt sind, welche Seite wird aus dem Hauptspeicher *verdrängt* (also auf einen Datenträger ausgelagert)?

Betrachtungsmerkmal:

- *Referenzkette*: charakterisiert konkreten Zugriffsvorgang über Angabe von Seitennummern, auf die nacheinander von einem Prozess zugegriffen wird

Referenzkette:		0	1	2	3	0	1	4	0	1	2	3	4
Hauptspeicher	Rahmen 0	0	0	0	0	0	0	0	0	0	0	0	4
	Rahmen 1		1	1	1	1	1	1	1	1	1	1	1
	Rahmen 2			2	2	2	2	2	2	2	2	2	2
	Rahmen 3				3	3	3	4	4	4	4	3	3

Zeit →

 Seitenfehler: virtuelle Seite nicht in Rahmen vorhanden

Behandlung von Seitenfehlern (5)

FIFO-Verdrängung

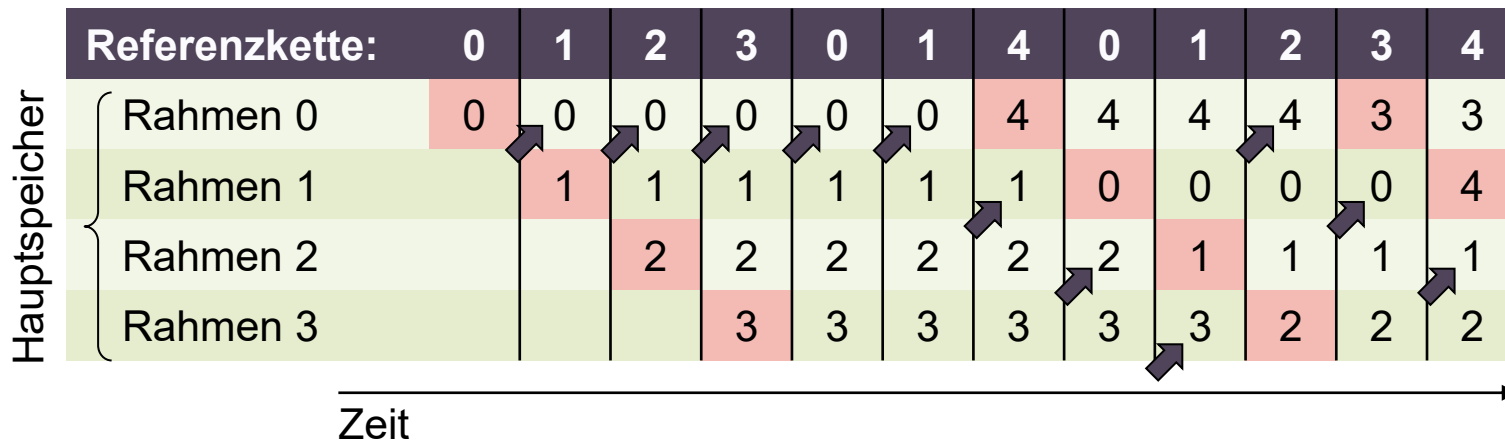
naiver Ansatz:

verdränge diejenige Seite, die schon am längsten im Hauptspeicher liegt

→ Implementierung mit *Clock-Algorithmus*: wandernder Zeiger auf älteste Seite

Nachteil:

- auch häufig benutzte Seiten werden regelmäßig ausgelagert
→ in der Praxis selten verwendet



Seitenfehler: virtuelle Seite nicht in Rahmen vorhanden

Behandlung von Seitenfehlern (6)

Belady-Verdrängung (1966)

theoretisch optimale Strategie:

verdränge immer diejenige Seite, die in Zukunft am längsten nicht gebraucht wird

Bewertung:

- verursacht prinzipiell eine minimale Anzahl an Seitenfehlern
- nötige Information ist oft im Voraus nicht oder nicht vollständig bekannt
- gut geeignet als Maßstab für andere Verdrängungsstrategien

Referenzkette:		0	1	2	3	0	1	4	0	1	2	3	4
Hauptspeicher	Rahmen 0	0	0	0	0	0	0	0	0	0	0	3	3
	Rahmen 1		1	1	1	1	1	1	1	1	1	1	1
	Rahmen 2			2	2	2	2	2	2	2	2	2	2
	Rahmen 3				3	3	3	4	4	4	4	4	4

Zeit →



Seitenfehler: virtuelle Seite nicht in Rahmen vorhanden

Behandlung von Seitenfehlern (7)

Beladys Anomalie

intuitive Einschätzung: umso mehr Rahmen es gibt, umso weniger Seitenfehler gibt es
→ *Beladys Anomalie*: das stimmt nicht immer!

Gegenbeispiel mit FIFO-Strategie:

- bei periodischem Durchlauf mit 3 Rahmen je einen Seitenfehler mehr als mit 3 Rahmen:

Referenzkette:	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen 0	0	0	0	3	3	3	4	4	4	4	4	4
Rahmen 1		1	1	1	0	0	0	0	0	2	2	2
Rahmen 2			2	2	2	1	1	1	1	1	3	3

mit 3 Rahmen:
9 Seitenfehler

Referenzkette:	0	1	2	3	0	1	4	0	1	2	3	4
Rahmen 0	0	0	0	0	0	0	4	4	4	4	3	3
Rahmen 1		1	1	1	1	1	1	0	0	0	0	4
Rahmen 2			2	2	2	2	2	2	1	1	1	1
Rahmen 3				3	3	3	3	3	3	2	2	2

mit 4 Rahmen:
10 Seitenfehler

Behandlung von Seitenfehlern (8)

Aging-Algorithmus

heuristischer Ansatz:

- speichere pro Seite in Zusatzbit (*R-Bit*, *Read-Bit*), ob auf die Seite zugegriffen wurde
- füge R-Bit regelmäßig (z. B. 20 ms) in Schieberegister pro Seite ein und setze R-Bit zurück
→ berücksichtigt letzte Zugriffszeit und Zugriffshäufigkeit

	Uhrtick 0	Uhrtick 1	Uhrtick 2	Uhrtick 3	Uhrtick 4
R-Bits pro Seite	A B C D E F 1 0 1 0 1 1	A B C D E F 1 1 0 0 1 0	A B C D E F 1 1 0 1 0 1	A B C D E F 1 0 0 0 1 0	A B C D E F 0 1 1 0 0 0
Register pro Seite	→ A 10000000 B 00000000 C 10000000 D 00000000 E 10000000 F 10000000	A 11000000 B 10000000 C 01000000 D 00000000 E 11000000 C 01000000	A 11100000 B 11000000 C 00100000 D 10000000 E 01100000 D 10100000	A 11110000 B 01100000 C 00010000 D 01000000 E 10110000 E 01010000	A 01111000 B 10110000 C 10001000 D 00100000 F 01011000 F 00101000

Zusammenfassung

Grundlagen

- getrennter Speicher von klein und schnell (Register) bis groß und langsam (Datenträger)
- Speichermodell aus Speicherzellen (Bytes) mit eindeutigen, fortlaufenden Adressen
- zu lösende Probleme: Fragmentierung, Relokation, Zusatzbelegung, Schutz

Virtueller Speicher

- virtueller Speicher als Lösung der vier Probleme der Speicherverwaltung
- jeder Prozess arbeitet mit eigenem, virtuellen Adressraum
- OS und CPU übersetzen mit Seitenabbildung in physische Adressen
 - Rahmen in physischem Adressraum erhalten dynamisch virtuelle Seiten zugeteilt
 - Verdrängungsstrategie legt fest, was passiert, wenn alle Rahmen schon belegt sind
 - Aging-Algorithmus verdrängt Seiten mit seltenem oder lange zurückliegenden Zugriff